

AI-powered bioinformatic analysis using R, #2

R Language Basics I

Wei-Hua Chen (CC BY-NC 4.0)

HUST, China

2026-05-27

Table of contents I

- 1 Data Types
- 2 Vectors
- 3 Matrices
- 4 Arrays
- 5 Basic Operators
- 6 Variables & Assignment
- 7 Vectorized Operations
- 8 Subsetting
- 9 Missing & Special Values
- 10 Summary & Homework



Basic Data Types

Every data structure in R is built from a few **atomic types**:

Type	Example	Description
numeric	3.14, 1e-3	Real numbers (double precision)
integer	287, 2L	Whole numbers (suffix L)
character	"hello", 'R'	Text strings
logical	TRUE, FALSE	Boolean values
complex	1+2i	Complex numbers (rarely used in bioinfo)

```
# integer
287
```

```
[1] 287
```

```
# decimal (double)
99.99
```

```
[1] 99.99
```

```
# scientific notation
```

Logical Values

Logical values are **TRUE** and **FALSE** (abbreviated T and F):

```
TRUE  
T  
FALSE  
F
```

They are **stored as numbers** internally:

```
1 + TRUE      # TRUE == 1
```

```
[1] 2
```

```
2 * FALSE     # FALSE == 0
```

```
[1] 0
```

Character Strings

Any text enclosed in single or double quotes:

```
'a sentence'           # single quotes  
"a string in Chinese"  # double quotes  
'1.123'                # looks like a number, but is a string  
'*%*( )!@##&@(9'      # gibberish is OK too
```

Creating Vectors

A **vector** is a one-dimensional sequence of values of the **same type**.

```
c(100, 20, 30)           # numeric vector
```

```
[1] 100  20  30
```

```
c("string", "array", "a vector")  # character vector
```

```
[1] "string"  "array"    "a vector"
```

```
c(TRUE, FALSE, TRUE, T, F)        # logical vector
```

```
[1]  TRUE FALSE  TRUE  TRUE FALSE
```

The `:` operator creates integer sequences:

```
2:8
```

```
[1] 2 3 4 5 6 7 8
```



Type Coercion Rules

A vector can only hold **one type**. When types are mixed, R coerces them:

Coercion order: logical $\rightarrow$ numeric $\rightarrow$ character

```
# logical + numeric  $\rightarrow$  all numeric  
class( c(45, TRUE, 20, FALSE, -100) )
```

```
[1] "numeric"
```

```
# logical + character  $\rightarrow$  all character  
str( c("string a", FALSE, "string b", TRUE) )
```

```
chr [1:4] "string a" "FALSE" "string b" "TRUE"
```

```
# numeric + character  $\rightarrow$  all character  
str( c("a string", 1.2, "another string", 1e-3) )
```

```
chr [1:4] "a string" "1.2" "another string" "0.001"
```



Vector Properties

```
x <- c(10, 100, 1000, 10000)
```

```
# Length of a vector  
length(x)
```

```
[1] 4
```

```
# Sum and mean  
sum(x)
```

```
[1] 11110
```

```
mean(x)
```

```
[1] 2777.5
```

Use `class()`, `str()`, and `typeof()` to inspect vectors — we'll compare them later.



Creating Matrices

A **matrix** is a 2-dimensional vector. Like vectors, all elements must be the **same type**.

```
matrix( c(20, 30.1, 2, 45.8, 23, 14), nrow = 2, byrow = T )
```

```
      [,1] [,2] [,3]
[1,] 20.0 30.1    2
[2,] 45.8 23.0   14
```

Key `matrix()` parameters:

Parameter	Meaning
<code>data</code>	Values to fill the matrix
<code>nrow</code>	Number of rows
<code>ncol</code>	Number of columns
<code>byrow</code>	Fill by row (TRUE) or by column (FALSE)
<code>dimnames</code>	Row and column names

matrix() byrow = TRUE vs FALSE

```
matrix( c(20, 30.1, 2, 45.8, 23, 14), nrow = 2, byrow = T )
```

```
      [,1] [,2] [,3]  
[1,] 20.0 30.1    2  
[2,] 45.8 23.0   14
```

```
matrix( c(20, 30.1, 2, 45.8, 23, 14), nrow = 2, byrow = F )
```

```
      [,1] [,2] [,3]  
[1,] 20.0  2.0  23  
[2,] 30.1 45.8  14
```

Named Rows & Columns

```
matrix( c(20, 30.1, 2, 45.8, 23, 14), nrow = 2,  
        dimnames = list( c("row_A", "row_B"), c("A", "B", "C") ) )
```

	A	B	C
row_A	20.0	2.0	23
row_B	30.1	45.8	14

Recycling & Truncation

When `nrow × ncol` `length(data)`, R recycles or truncates **silently**:

```
# data length 20, matrix size 10 → data is truncated
```

```
matrix( 1:20, nrow = 2, ncol = 5, byrow = T)
```

```
      [,1] [,2] [,3] [,4] [,5]
[1,]     1     2     3     4     5
[2,]     6     7     8     9    10
```

```
# data length 3, matrix size 6 → data is recycled
```

```
matrix( 1:3, nrow = 2, ncol = 3, byrow = T )
```

```
      [,1] [,2] [,3]
[1,]     1     2     3
[2,]     1     2     3
```

Recycling Warnings

Warning 1: matrix larger than data, but not an integer multiple:

```
matrix( 1:3, nrow = 2, ncol = 4, byrow = T )
```

	[,1]	[,2]	[,3]	[,4]
[1,]	1	2	3	1
[2,]	2	3	1	2

Warning 2: matrix smaller than data, but not an integer multiple:

```
matrix( letters[1:20], nrow = 3, ncol = 5, byrow = T )
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	"a"	"b"	"c"	"d"	"e"
[2,]	"f"	"g"	"h"	"i"	"j"
[3,]	"k"	"l"	"m"	"n"	"o"

Matrix Properties

```
m <- matrix( c(20, 30.1, 2, 45.8, 23, 14), nrow = 2, byrow = T )
```

```
dim(m)      # dimensions
```

```
[1] 2 3
```

```
nrow(m)     # number of rows
```

```
[1] 2
```

```
ncol(m)     # number of columns
```

```
[1] 3
```

```
range(m)    # min and max
```

```
[1]  2.0 45.8
```

```
summary(m)  # 5-number summary
```

V1	V2	V3
Min. :20.00	Min. :23.00	Min. : 2
1st Qu.:26.45	1st Qu.:24.77	1st Qu.: 5
Median :32.90	Median :26.55	Median : 8



class() vs str()

```
class( matrix( c(20, 30.1, 2, 45.8, 23, 14),  
              nrow = 2, byrow = T ) )
```

```
[1] "matrix" "array"
```

```
str( matrix( c(20, 30.1, 2, 45.8, 23, 14),  
            nrow = 2, byrow = T ) )
```

```
num [1:2, 1:3] 20 45.8 30.1 23 2 14
```

- `class()` — returns the class name (e.g., "matrix", "array")
- `str()` — shows the **structure**: type, dimensions, first few values

Multi-dimensional Arrays

Vectors are 1D, matrices are 2D. **Arrays** generalize to any number of dimensions:

```
array( data = LETTERS[1:16],
      dim  = c(2, 4, 2),
      dimnames = list( c("A","B"),
                       c("one","two","three","four"),
                       c("first", "second") ) )
```

, , first

	one	two	three	four
A	"A"	"C"	"E"	"G"
B	"B"	"D"	"F"	"H"

, , second

	one	two	three	four
A	"I"	"K"	"M"	"O"
B	"J"	"L"	"N"	"P"

- All elements must be the **same type**
- `dim` specifies the shape (rows, columns, layers, ...)

Arithmetic Operators

```
1 + 2 - 3 * 4 / 5      # +, -, *, /  
1 + (2 - 3) * 4 / 5    # parentheses change precedence
```

```
2 ^ 6                  # exponentiation (power)  
5 %% 2                 # modulo (remainder)
```

Logical Operators

```
T | F          # OR — TRUE if either is TRUE
T & F          # AND — TRUE only if both are TRUE
5 | 0          # non-zero = TRUE, zero = FALSE
```

```
5 & -1         # what do you think this returns?
```

```
[1] TRUE
```

Math Functions

Commonly used math functions:

Function	Description
<code>log(x)</code>	Natural logarithm
<code>log2(x)</code> , <code>log10(x)</code>	Base-2, base-10 log
<code>exp(x)</code>	Exponential (e^x)
<code>sqrt(x)</code>	Square root
<code>sin(x)</code> , <code>cos(x)</code>	Trigonometry
<code>abs(x)</code>	Absolute value
<code>round(x, digits=n)</code>	Round to n decimal places

Assignment Operators

R uses `<-` for assignment (the recommended style):

```
x <- c(10, 100, 1000, 10000)  # recommended
x = c(10, 100, 1000, 10000)  # also works
c(10, 100, 1000, 10000) -> x  # rarely used (rightward)
assign("x", c(10, 100, 1000, 10000)) # functional form
```



Tip

After assignment, the value is stored silently (not printed). Enclose in `()` to print: `(x <- 1:5)`.

Managing Variables

```
x <- c(10, 100, 1000, 10000)
m <- matrix( c(20, 30.1, 2, 45.8, 23, 14), nrow = 2,
             dimnames = list( c("row_A", "row_B"), c("A", "B", "C") ) )
```

```
ls()                                # list all variables in current environment
```

```
[1] "m"                                "pandoc_dir"      "quarto_bin_path" "x"
```

```
rm(x)                                # remove one variable
ls()
```

```
[1] "m"                                "pandoc_dir"      "quarto_bin_path"
```

```
# rm(list = ls())                  # remove ALL variables — use with caution!
```

Viewing Variables in RStudio

RStudio's **Environment** pane (top-right) shows all current variables:

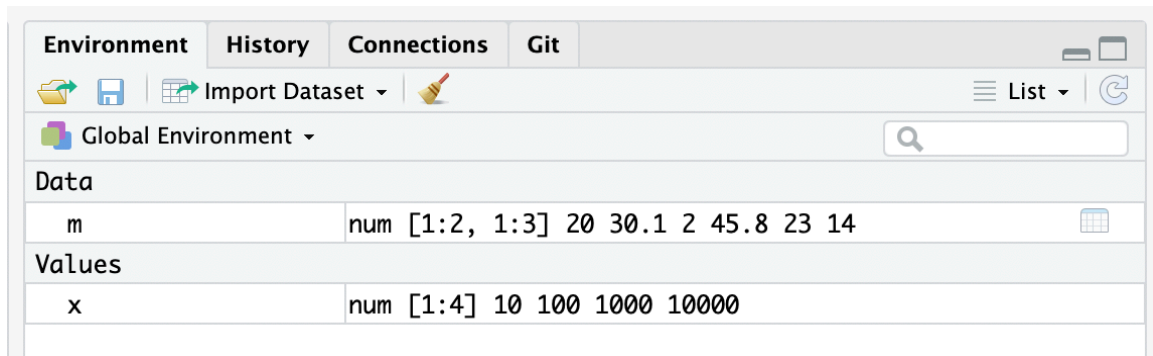


Figure 1: RStudio Environment window

Vectorization — The Key R Paradigm

Operations are automatically applied **element-by-element** — no loops needed:

```
x <- c(10, 100, 1000, 10000)
( y <- sqrt(x) * 4 + 10 )
```

```
[1] 22.64911 50.00000 136.49111 410.00000
```

Every operation above (`sqrt`, `*`, `+`) is applied to **each element** of `x` simultaneously.

Recycling Rules

When two vectors have **different lengths**, the shorter one is **recycled**:

```
# Shorter vector recycled to match longer one
x / c(10, 100)      # c(10,100) → c(10,100,10,100)
```

```
[1]  1   1 100 100
```

```
# Incomplete recycling produces a warning but still runs
x / c(10, 100, 1000)  # longer vector length 3, shorter recycled
```

```
[1]  1    1    1 1000
```


Matrix Operations

Matrix arithmetic also follows vectorization:

```
m
```

```
      A      B      C
row_A 20.0   2.0  23
row_B 30.1  45.8  14
```

```
m / 10                                # divide every element by 10
```

```
      A      B      C
row_A 2.00  0.20  2.3
row_B 3.01  4.58  1.4
```

```
m / c(1, 10, 100)                   # recycle across columns (column-major)
```

```
      A      B      C
row_A 20.00  0.02  2.30
row_B  3.01 45.80  0.14
```

```
m / c(1, 10)                         # recycle across columns
```

```
      A      B      C
row_A 20.00  2.00  23.0
```

Vector Subsetting with []

Extract elements by **position**, **name**, or **logical condition**:

```
ab <- c("a", "b", "c", "d", "e", "f")
```

```
ab[1]                                # single element
```

```
[1] "a"
```

```
ab[ c(1, 4, 5) ]                    # multiple by position
```

```
[1] "a" "d" "e"
```

```
ab[ 2:5 ]                            # range
```

```
[1] "b" "c" "d" "e"
```

```
ab[ length(ab) ]                    # last element
```

```
[1] "f"
```

Vector — Replace Values

```
ab[1] <- "A"           # replace single element  
ab
```

```
[1] "A" "b" "c" "d" "e" "f"
```

```
ab[c(2, 3)] <- c("B", "C") # replace multiple  
ab
```

```
[1] "A" "B" "C" "d" "e" "f"
```

Vector — Named Elements

```
names(ab) <- ab          # name elements after themselves
ab
```

```
  A    B    C    d    e    f
"A" "B" "C" "d" "e" "f"
```

```
ab[ c("A", "B") ] <- c("alpha", "beta")  # replace by name
ab
```

```
      A      B      C      d      e      f
"alpha" "beta" "C"   "d"   "e"   "f"
```

Matrix Subsetting — Rows

Matrix subsetting uses [row, column]:

```
m
```

```
      A    B  C
row_A 20.0  2.0 23
row_B 30.1 45.8 14
```

```
m[1, ]           # first row
```

```
  A  B  C
20  2 23
```

```
m[1:2, ]         # first two rows
```

```
      A    B  C
row_A 20.0  2.0 23
row_B 30.1 45.8 14
```

```
m["row_A", ]     # by row name
```

```
  A  B  C
20  2 23
```

```
m[c("row_B", "row_A"), ] # by name, custom order
```

Matrix Subsetting — Columns

```
m[, 1] # first column
```

```
row_A row_B
20.0 30.1
```

```
m[, c(1, 2)] # first two columns
```

```
      A      B
row_A 20.0  2.0
row_B 30.1 45.8
```

```
m[, c("B", "A")] # by column name, custom order
```

```
      B      A
row_A  2.0 20.0
row_B 45.8 30.1
```

Matrix Subsetting — Sub-blocks

```
m[1:2, 2:3]           # rows 1-2, columns 2-3
```

```
      B  C  
row_A 2.0 23  
row_B 45.8 14
```

Matrix — Replace Values

```
# Replace an entire row
m[1, ] <- c(10)
m
```

```
      A      B      C
row_A 10.0 10.0 10
row_B 30.1 45.8 14
```

```
# Replace an entire column
m[, "C"] <- c(230, 140)
m
```

```
      A      B      C
row_A 10.0 10.0 230
row_B 30.1 45.8 140
```


Matrix Transpose

```
t(m)           # swap rows and columns
```

	row_A	row_B
A	10	30.1
B	10	45.8
C	230	140.0

Other Useful Vector Functions

```
rev(1:10)                # reverse order
```

```
[1] 10  9  8  7  6  5  4  3  2  1
```

```
lts <- sample( LETTERS[1:20] )
sort(lts)                # sort values
```

```
[1] "A" "B" "C" "D" "E" "F" "G" "H" "I" "J" "K" "L" "M" "N" "O" "P" "Q" "R" "S"
[20] "T"
```

```
order(lts)                # indices of sorted positions
```

```
[1]  8  5 17  2 16 19  9 12 13 14  6  4 11  7  1 15 10 18  3 20
```

R Data Structures Overview

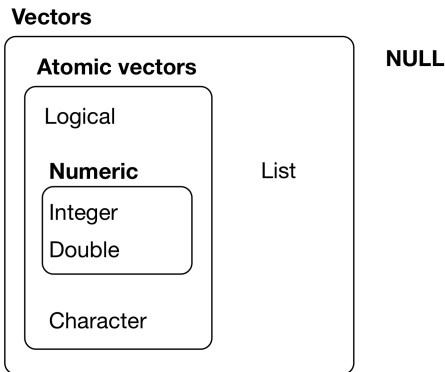


Figure 2: R data structures

Image from: <https://r4ds.had.co.nz/vectors.html>

typeof() — The Fundamental Type

```
typeof(letters)           # character
```

```
[1] "character"
```

```
typeof(1:10)             # integer
```

```
[1] "integer"
```

```
typeof( list("a", "b", 1:10) ) # list
```

```
[1] "list"
```

```
typeof( 1:10 %% 3 == 0 )  # logical
```

```
[1] "logical"
```

```
typeof(1)                # double
```

```
[1] "double"
```

```
typeof(1L)               # integer (note the L suffix)
```

```
[1] "integer"
```

Special Values

R has several special values for edge cases:

Value	Meaning	Example
NA	Not Available (missing)	<code>c(1, NA, 3)</code>
NaN	Not a Number	<code>0 / 0</code>
Inf / -Inf	Infinity	<code>1 / 0</code>
NULL	Undefined / empty	Returned by some functions

```
c(-1, 0, 1) / 0           # produces -Inf, NaN, Inf
```

```
[1] -Inf  NaN  Inf
```

is.*() Test Functions

Instead of comparing with `== NA`, use `is.*()` functions:

```
# Read a small CSV with missing values
nul <- readr::read_csv(file = "data/talk02/null.csv")
library(kableExtra)
kbl(nul, booktabs = T)
```

function	0	-Inf	NA	NaN
<code>is.finite()</code>	x	NA	NA	NA
<code>is.infinite()</code>	NA	x	NA	NA
<code>is.na()</code>	NA	NA	x	x
<code>is.nan()</code>	NA	NA	NA	x

More is.*() Functions

```
is.null(NULL)           # TRUE
is.numeric(NA)          # TRUE (NA inherits numeric type!)
is.numeric(Inf)         # TRUE
```

Type-checking functions

```
is.list(x)
is.logical(x)
is.character(x)
is.vector(x)
is.matrix(m)
is.array(x)
```

Many more...

Today's Summary — Data Types

Category	What we covered
Atomic types	numeric, integer, character, logical, complex
Coercion	logical $\rightarrow$ numeric $\rightarrow$ character
Vectors	<code>c()</code> , <code>:</code> , single-type constraint
Matrices	<code>matrix()</code> , <code>nrow/ncol</code> , <code>byrow</code> , <code>dimnames</code>
Arrays	<code>array()</code> , <code>dim</code> , multi-dimensional

Today's Summary — Operations

Category	What we covered
Operators	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>^</code> , <code>%%</code> , <code> </code> , <code>&</code>
Variable mgmt	<code><-</code> , <code>=</code> , <code>ls()</code> , <code>rm()</code>
Vectorization	element-wise operations, recycling rules
Subsetting	<code>[]</code> by position/name/condition
Special values	<code>NA</code> , <code>NaN</code> , <code>Inf</code> , <code>NULL</code> , <code>is.*()</code>

Today's Summary — Key Functions

Function	Purpose
<code>class()</code> , <code>str()</code> , <code>typeof()</code>	Inspect object type/structure
<code>length()</code> , <code>dim()</code> , <code>nrow()</code> , <code>ncol()</code>	Dimensions
<code>sum()</code> , <code>mean()</code> , <code>range()</code> , <code>summary()</code>	Summary statistics
<code>sort()</code> , <code>order()</code> , <code>rev()</code>	Ordering
<code>names()</code> , <code>dimnames()</code>	Naming
<code>t()</code>	Matrix transpose
<code>is.null()</code> , <code>is.na()</code> , <code>is.numeric()</code> , ...	Type tests

Homework

- 1 Open `Exercises` and `homework/talk02-homework.Rmd` in your course repository
- 2 Complete all exercises — use CodeBuddy AI for help!
- 3 Render the `.Rmd` to PDF
- 4 Submit as instructed on the course platform



Tip

Pro tip: When stuck, ask CodeBuddy: “Explain this R error to me: [paste error]” or “Help me understand how vector recycling works in R.”

Next Talk

Talk 03 — R Language Basics II

- Factors: creation, levels, ordering (`forcats`)
- Lists: creation, naming, subsetting
- Data frames vs tibbles
- Data import/export with `readr`
- Introduction to tidy data